Array, stringhe e matrici

Marco Cerbella

Lezione 15 – 2 novembre 2026

1 Array

Un *array* contiene oggetti di un dato tipo, memorizzati consecutivamente in un blocco continuo di memoria. I singoli oggetti sono detti *elementi* dell'array. Il tipo degli elementi può essere qualunque tipo di oggetto.

Il tipo di un array è determinato dal tipo e dal numero degli elementi. Se gli elementi hanno tipo T, l'array è detto «array di T». Se gli elementi hanno tipo `int`, per esempio, il tipo dell'array è «array di `int`». Il tipo è però *incompleto*, a meno che non si specifichi anche il numero di elementi: un array di `int` con 16 elementi ha un tipo di oggetto completo, «array di 16 elementi `int`».

1.1 Operatore di indicizzazione

L'operatore di indicizzazione `[]` permette di accedere ai singoli elementi di un array. Ha due operandi. Nel caso più semplice un operando è il nome di un array e l'altro è un intero. L'espressione `myarray[i]` designa allora l'elemento numero i dell'array, dove il primo elemento ha numero zero. L'operando sinistro di `[]` non deve necessariamente essere il nome di un array; l'operando destro deve avere tipo intero.

1.2 Definizione di array

È un tipo *aggregato*: una sequenza di oggetti con lo stesso tipo. La definizione di un array determina il suo nome, il tipo dei suoi elementi e il numero degli elementi. Una definizione di array senza inizializzazione esplicita ha la sintassi

```
tipo nome[ numero_di_elementi ];
```

Il numero di elementi, tra parentesi quadre, deve essere un'espressione intera con valore maggiore di zero. Per esempio:

```
1 char buffer[4*512];
```

Questa riga definisce un array di nome `buffer` formato da 2048 elementi di tipo `char`; `sizeof(buffer)` vale $2048 \cdot$ `sizeof(char)`.

1.3 Array a lunghezza variabile

Dallo standard C99 è possibile definire un array usando per il numero di elementi un'espressione non costante (*variable-length array*, VLA). Dal C11 sono una caratteristica *opzionale* dei compilatori: alcuni compilatori potrebbero non implementarli.

```c
void func(int n)
{
    int vla1[2*n];
    int vla2[n];
    /* ... */
}
```

1.4 Accesso agli elementi

L'espressione `myArray[i]` designa l'elemento dell'array con indice i. Gli elementi sono indicizzati a partire da 0: se `len` è il numero di elementi di un array, l'ultimo elemento ha indice `len` -1.

```c
const int a = 4;
long mArray[a];
for (int i = 0; i < a; ++i)
    mArray[i] = 2 * i;
```

mArray[0]	mArray[1]	mArray[2]	mArray[3]
0	2	4	6

1.5 Inizializzazione

Per inizializzare esplicitamente un array quando lo si definisce, si usa una *lista di inizializzazione*: una lista di inizializzatori separati da virgole.

```c
int a[4] = { 1, 2, 4, 8 };   // a[0] = 1, a[1] = 2, a[2] = 4, a[3] = 8
```

Esempio (Accesso fuori dai limiti)**.**

```c
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

for (int i = 0; i < 20; i++) {
    printf("Element[%d] = %d\n", i+1, a[i]);
}
```

I primi 10 elementi stampati sono $1, \ldots, 10$; dall'undicesimo in poi si leggono valori casuali (per esempio 1339107492, 2097207605, 32767, ...): il ciclo accede a locazioni *oltre la fine dell'array*. In C non c'è nessun controllo sui limiti: il comportamento è indefinito.

Regole per l'inizializzazione

Quando si inizializza un array si devono rispettare le regole seguenti.

- Non si può includere un'inizializzazione nella definizione di un array a lunghezza variabile.
- Si può omettere la lunghezza dell'array nella definizione se si fornisce una lista di inizializzazione: la lunghezza è allora determinata dall'indice dell'ultimo elemento per il quale la lista contiene un inizializzatore.

- Se la definizione contiene sia una lunghezza sia una lista di inizializzazione, la lunghezza è quella specificata dall'espressione tra le parentesi quadre. Gli elementi per cui nella lista non c'è un inizializzatore vengono inizializzati a zero. Se la lista contiene più inizializzatori di quanti elementi ha l'array, quelli in eccesso vengono semplicemente ignorati.
- L'ultima virgola nella lista degli inizializzatori viene ignorata.

Di conseguenza tutte le definizioni seguenti sono equivalenti:

```
int a[4] = { 1, 2 };
int a[]  = { 1, 2, 0, 0 };
int a[]  = { 1, 2, 0, 0, };
int a[4] = { 1, 2, 0, 0, 5 };    // l'inizializzatore 5 è ignorato
```

Nell'ultima definizione l'inizializzatore 5 è ignorato; la maggior parte dei compilatori genera un warning quando c'è questo disallineamento. Se gli inizializzatori non hanno lo stesso tipo degli elementi, avviene una conversione implicita di tipo.

Inizializzare elementi specifici

I *designatori di elemento* permettono di associare gli inizializzatori a elementi specifici. Per specificare l'elemento da inizializzare si mette il suo indice tra parentesi quadre: `[espressione costante]`.

```
int a[6] = { 1, 2, [3] = 1, 2 };   // equivale a { 1, 2, 0, 1, 2, 0 }
```

Questa definizione inizializza gli elementi `a[0]` e `a[3]` con il valore 1 e gli elementi `a[1]` e `a[4]` con il valore 2. Tutti gli altri elementi ricevono il valore iniziale 0.

Il designatore deve essere un'*espressione costante intera*:

```
int three = 2;
int a[4] = { 1, [three] = 1 };
// error: expression is not an integer constant expression
```

```
const int three = 2;
int a[4] = { 1, [three] = 1 };
// ok (con alcuni compilatori): a = { 1, 0, 1, 0 }
```

1.6 Array come parametri di funzione

Esempio.

```
#include <stdio.h>
void fun(int a[3]) {
    printf("%lu\n", sizeof(a));
    for (int i = 0; i < 3; ++i)
        a[i] = i;
```

```c
}
int main() {
    int a[3];
    printf("%lu\n", sizeof(a));
    fun(a);
}
```

```
12
8
```

Nella `main`, `sizeof(a)` vale 12 ($3 \cdot 4$ byte). Nella funzione, invece, il parametro `int a[3]` è in realtà un puntatore: `sizeof(a)` restituisce la dimensione di `int *` (8 byte), e il compilatore avverte con `[-Wsizeof-array-argument]`: «sizeof on array function parameter will return size of 'int *' instead of 'int [3]'».

2 Stringhe

Una *stringa* è una sequenza continua di caratteri terminata da `'\0'`, il carattere nullo. La lunghezza di una stringa è il numero dei caratteri escluso il carattere nullo di terminazione. Le stringhe sono memorizzate in array i cui elementi hanno tipo `char`.

Si possono inizializzare array di `char` con literal stringa. Per esempio le due definizioni seguenti sono equivalenti (lunghezza della stringa 8, lunghezza dell'array 30):

```c
char str1[30] = "Let's go";
char str1[30] = { 'L', 'e', 't', '\'', 's', ' ', 'g', 'o', '\0' };
```

Un array che contiene una stringa deve essere sempre lungo almeno un elemento in più della lunghezza della stringa, per far posto al carattere nullo di terminazione.

Esempio (Array troppo corto)**.**

```c
#include <stdio.h>
int main() {
    char string[5] = "Jim Morrison";
    printf("%s\n", string);
}
```

Il compilatore avverte `initializer-string for char array is too long`; l'output è composto da caratteri casuali dopo `Jim M`, perché l'array non contiene il carattere nullo di terminazione.

Esempio (Carattere nullo nel mezzo)**.**

```c
#include <stdio.h>
int main() {
    char string[14] = {'J','i','m',' ','M','\0','o','r','r','i','s','o','n','\0'
    };
```

```
4       printf("%s\n", string);
5   }
```

```
Jim M
```

La stampa si ferma al primo `'\0'`.

Se si definisce un array di caratteri senza lunghezza esplicita e lo si inizializza con un literal stringa, l'array creato è lungo un elemento *in più* della lunghezza della stringa:

```
1   char str2[] = " to London!";    // lunghezza della stringa: 11
2                                   // lunghezza dell'array: 12
```

3 Array multidimensionali

Un array multidimensionale in C è semplicemente un array i cui elementi sono a loro volta array. Gli elementi di un array a n dimensioni sono array a $(n-1)$ dimensioni. Per esempio, ogni elemento di un array a due dimensioni è un array a una dimensione.

La dichiarazione di un array multidimensionale ha una coppia di parentesi quadre per ogni dimensione:

```
1   char screen[10][40][80];    // un array a tre dimensioni
```

Ha 10 elementi, da `screen[0]` a `screen[9]`; ognuno è un array a due dimensioni, formato a sua volta da 40 array a una dimensione di 80 caratteri. In totale $10 \cdot 40 \cdot 80 = 32\,000$ elementi. Lo standard raccomanda alle implementazioni di accettare almeno 256 dimensioni (ISO 14882, B.2), ma possono supportarne meno o più.

3.1 Matrici

Gli array bidimensionali sono detti anche *matrici*. Sono molto usati e meritano uno sguardo più attento. La matrice `mat` della definizione seguente ha tre righe e cinque colonne (conviene ragionare in termini di righe e colonne):

```
1   float mat[3][5];
```

`mat[0]`, `mat[1]` e `mat[2]` sono le righe della matrice; ognuna è un array di cinque elementi `float`. In totale 15 elementi `float`.

```
1   for (int row = 0; row < 3; ++row)
2       for (int col = 0; col < 5; ++col)
3           mat[row][col] = row + (float) col / 10;
```

3.2 Inizializzazione

Si possono inizializzare array multidimensionali con una lista di inizializzazione secondo le regole già viste:

```
1 int a3d[2][2][3] = { { { 1, 0, 0 }, { 4, 0, 0 } }, { { 7, 8, 0 }, { 0, 0, 0 } }
      };
2 int a3d[][2][3]  = { { { 1 }, { 4 } }, { { 7, 8 } } };
```

Nella seconda non occorre specificare che la prima dimensione ha ampiezza 2, perché la lista più esterna contiene due inizializzatori.

Si possono anche omettere alcune parentesi graffe. Se una coppia di graffe contiene più inizializzatori del numero di elementi della dimensione corrispondente, quelli in eccesso sono associati all'elemento successivo nella sequenza di memorizzazione:

```
1 int a3d[2][2][3] = { { 1, 0, 0, 4 }, { 7, 8 } };
2 int a3d[2][2][3] = { 1, 0, 0, 4, 0, 0, 7, 8 };
```

Si possono usare anche i designatori di elemento:

```
1 int a3d[2][2][3] = { 1, [0][1][0] = 4, [1][0][0] = 7, 8 };
```

4 Riferimenti

Sul libro: sezioni 6.1–6.5, 6.11; stringhe: capitoli 8 e 9.